

Tema 2

Sintaxis básica de C#



Índice

- Introducción
- Estructura de un programa en C#
- Comentarios en C#
- Variables y tipos de datos primitivos
- Conversión de tipos
- Operadores
- Cadenas de texto
- Trabajando con la consola
- Código de estilo



- En este tema vamos a aprender los conceptos básicos para crear nuestros primeros programas sencillos en C#.
- Aprenderemos la estructura básica de estos, qué son las variables, los diferentes tipos primitivos que utilizaremos para almacenar los datos en nuestras variables y las operaciones básicas entre ellos.

1. Introducción



- Haremos hincapié en las cadenas de texto y sus operaciones y aprenderemos a crear aplicaciones interactivas mostrando datos por pantalla y solicitando datos al usuario mediante teclado.
- Por último, veremos un código de estilo que nos ayudará a que nuestros programas sean más legibles, robustos y mantenibles.

1. Introducción

- Un programa básico de C# sigue una estructura típica que consta de:
 - *usings*
 - *namespace*
 - Clase
 - Variables
 - Métodos
 - Sentencias

2. Estructura de un programa en C#

2. Estructura de un programa en C#

```
1  using System;
2
3  namespace ConsoleApp1
4  {
5      internal class Program
6      {
7          static void Main(string[] args)
8          {
9              Console.WriteLine("Hello, World!")
10          }
11      }
```

El programa por defecto que Visual Studio nos crea contiene el siguiente código:

- `using` es una directiva que nos permite **importar tipos y métodos** de librerías externas a nuestro código.
- La librería `System` forma parte de la BCL y nos ofrece funcionalidad básica como el trabajo con la consola. Además, contiene otros subpaquetes como `System.Math` para realizar operaciones matemáticas complejas.

Para utilizarlo habría que importarlo mediante otro `using`.

**En la versión 13 de C#, el `import` de `System` se hace de manera implícita, no es necesario su `using` explícito.*

2. Estructura de un programa en C#

- namespace es una palabra clave que se utiliza para declarar el **ámbito o área lógica** en el que se organiza una clase.
- Los *namespaces* o espacios de nombres suelen contener diferentes clases relacionadas entre si.
- Además de para organizar el código, también sirven para evitar conflictos de nomenclatura en diferentes partes de un programa.

2. Estructura de un programa en C#

- `class` es la palabra clave con la que se declara una clase, seguida del nombre identificativo que va a tener.
- Como vimos en el tema anterior, las clases son las unidades básicas en la **programación orientada a objetos**.
- Aprenderemos más sobre las clases y las partes que las componen en el tema 7.

2. Estructura de un programa en C#

- En el código de ejemplo, la clase no contiene ninguna variable.
- `Main` es el único método existente. Como norma general, todos los programas tienen como **punto de entrada** el método *Main*, por lo que será el primero que se ejecute y el que gestione el flujo de la aplicación.
- Presenta una serie de palabras clave como el modificador de acceso (`internal`) o la inclusión de parámetros (`string[] args`) que veremos más adelante qué significan.

2. Estructura de un programa en C#

- Dentro de un método encontramos las **sentencias** que definen su comportamiento.
- En el ejemplo hay una única sentencia `Console.WriteLine("Hello, World!");`. Esta mostrará por consola el texto *Hello, World!*. La ejecución continuaría acto seguido por la siguiente sentencia si la hubiese.
- Las sentencias en C# siempre deben acabar con el carácter `;` (punto y coma).

2. Estructura de un programa en C#

- En el código de ejemplo también aparecen líneas que únicamente contienen llaves ({ }).
- Estas llaves se utilizan para definir el alcance de un bloque de código, como una clase, un método o una estructura de control.
- Todas las llaves abiertas deben ser cerradas. Se pueden anidar bloques dentro de otros bloques (por ejemplo, el namespace contiene una clase y esta a su vez contiene un método, cada uno delimitado con sus llaves).

2. Estructura de un programa en C#

- En las últimas versiones de .NET existe lo que se conoce como *instrucciones de nivel superior*.

En ella, la clase Program únicamente contiene un comentario y la sentencia interna de consola, sin necesidad de declarar ninguno de los elementos vistos con anterioridad (namespace, class, métodos...).

- Nosotros vamos a utilizar la estructura clásica a lo largo del curso para asentar las bases y fundamentos del lenguaje.

2. Estructura de un programa en C#

```
1 // See https://aka.ms/new-console-template for more
```

```
2 Console.WriteLine("Hello, World!");
```

- Los comentarios son bloques de texto que no se tienen en cuenta a la hora de compilar el código, a nivel funcional son como si no existieran.
- Es una herramienta que nos ayuda a hacer el código más comprensible, especialmente si volvemos pasado un tiempo o si compartimos proyecto entre varios colaboradores.

3. Comentarios en C#

▸ Ejemplo:

3. Comentarios en C#

```
1 // Mostramos en consola una suma
2 Console.WriteLine(5+8); // El programa escribirá '13'
3 /*
4     Ahora vamos a escribir una cadena
5     por consola.
6     En este caso hay que poner
7     el texto entre comillas.
8 */
9 Console.WriteLine("Curso C# 13");
10
11 Console.WriteLine(7 /* Vamos a restar */ - 2);
```

- Todo el texto que aparece en gris oscuro son comentarios que no afectan al comportamiento del programa.
- En C# existen 2 tipos principales de comentarios: de línea única o de bloque.
- Existe también un tercer tipo de comentario que sirve para generar documentación XML, aunque no lo vamos a ver de momento.

3. Comentarios en C#

Comentarios de línea única

- Los comentarios de línea única comienzan con doble barra (//) y se extienden hasta el final de la línea.
- Pueden comenzar al inicio de una línea (línea 1 en el código de ejemplo) o después de código ejecutable (línea 2), pero una vez que comienzan, toda la línea restante es comentario y no puede haber código ejecutable detrás.
- Son útiles para hacer anotaciones cortas o para deshabilitar líneas de código sin tener que borrarlas.

3. Comentarios en C#

Comentarios de bloque

- Los comentarios de bloque comienzan con `/*` y finalizan con `*/`.
- Pueden ocupar varias líneas (líneas 3-8 en el código de ejemplo) o estar intercalados entre código ejecutable (línea 11).
- Son útiles para hacer anotaciones más extensas o para deshabilitar bloques de código grandes sin tener que borrarlos.

3. Comentarios en C#

Variables

- Una **variable** se define en C# como un espacio de memoria donde se **almacena un valor** que puede ser utilizado o modificado en la ejecución del programa.
- Las variables nos permiten almacenar datos de distintos tipos y trabajar con ellos.
- En C#, todas las variables deben ser **declaradas** antes de poder utilizarse. Esto significa que deben tener un tipo y un nombre.

4. Variables y tipos de datos primitivos

- Ejemplo:
- En este ejemplo tenemos 3 variables de diferente tipo.
- Todas las variables se **declaran** especificando su tipo y su nombre.
- Una vez declaradas, se les puede asignar valores.

4. Variables y tipos de datos primitivos

```
1 string name;  
2 name = "Manuel";  
3 int age = 36;  
4 bool isMale = true;
```

- La **asignación** de valores a una variable en C# se realiza mediante el símbolo =.
- La asignación se puede realizar en la misma línea de la declaración (líneas 3 y 4 del código de ejemplo) o con posterioridad (línea 2).
- El tipo de dato de la variable se especifica únicamente en su declaración (líneas 1, 3 y 4). Para utilizar la variable posteriormente únicamente utilizamos su nombre (línea 2).

4. Variables y tipos de datos primitivos

- También podemos declarar variables utilizando la palabra clave `var` si van seguidos de un valor o expresión.
- En ese caso, el compilador infiere el tipo de variable a partir de la expresión de inicialización.
- Un uso común de la palabra clave `var` es cuando llamemos a constructores de una clase. Veremos esto en el tema 7.

4. Variables y tipos de datos primitivos

Ejemplo

- En ese ejemplo, la variable `name` es de tipo `string`, `isEnabled` de tipo `bool`, `count` de tipo `int` y `price` de tipo `float`. Veremos cada uno de estos tipos a continuación.

4. Variables y tipos de datos primitivos

```
1 var name = "Antonio";  
2 var isEnabled = true;  
3 var count = 20;  
4 var price = 0.00f;
```

- Los nombres de las variables pueden contener letras, números (pero no pueden ir al principio), guion bajo (_) y arroba (@, únicamente al principio).
- Los nombres de las variables distinguen entre mayúsculas y minúsculas. Esto quiere decir que la variable name sería una variable diferente a la variable Name.
- En el siguiente tema veremos conceptos y usos avanzados de las variables.

4. Variables y tipos de datos primitivos

Tipos de datos primitivos

- En C# denominamos tipos primitivos de datos a los tipos de datos simples que pueden almacenar las variables.
- Los tipos primitivos están integrados en el framework y se componen de diferentes tipos numéricos, un tipo carácter de texto y un tipo booleano lógico (verdadero/falso).

4. Variables y tipos de datos primitivos

Tipos numéricos enteros

- Los tipos numéricos enteros representan números sin decimales.
- Dependiendo del rango que puedan almacenar y si admiten números negativos o no, tenemos diferentes tipos primitivos para trabajar con números enteros.
- En C# tenemos 8 tipos numéricos enteros: `int`, `long`, `short`, `sbyte`, `uint`, `ulong`, `ushort` y `ubyte`.

4. Variables y tipos de datos primitivos

Tipos numéricos enteros

4. Variables y tipos de datos primitivos

Tipo	Tamaño	Rango	Negativos
int	32 bits	-2,147,483,648 a 2,147,483,647	Sí
long	64 bits	-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807	Sí
short	16 bits	-32,768 a 32,767	Sí
sbyte	8 bits	-128 a 127	Sí
uint	32 bits	0 a 4,294,967,295	No
ulong	64 bits	0 a 18,446,744,073,709,551,615	No
ushort	16 bits	0 a 65,535	No

byte	8 bits	0 a 255	No
------	--------	---------	----

int

- Es el tipo más común para representar números enteros en C#.
- Ofrece un buen equilibrio entre rango y tamaño, ya que nos proporciona un rango extenso y ocupa poco tamaño en memoria.

4. Variables y tipos de datos primitivos

```
1 int number = 4723
```

long

- Se utiliza para almacenar números muy grandes que superan o pueden superar el rango de `int`.
- Una variable de tipo `long` ocupa el doble de espacio en memoria que una variable de tipo `int`.

4. Variables y tipos de datos primitivos

```
1 long bigNumber = 1201627261
```

short

- Se utiliza para optimizar el almacenamiento en memoria cuando sabemos que el dato no va a contener números muy grandes.

4. Variables y tipos de datos primitivos

```
1 short shortNumber = -
```

sbyte

- Se utiliza para optimizar mucho el almacenamiento en memoria cuando sabemos que el dato va a contener únicamente números muy pequeños (de -128 a 127 sólo).

4. Variables y tipos de datos primitivos

```
1 sbyte veryShortNumber
```

Tipos 'unsigned'

- Los tipos *unsigned* (sin signo) son versiones de los tipos que hemos explicado que solamente admiten números positivos (ver tabla). Cada tipo primitivo entero tiene su equivalente *unsigned*.
- Se utilizan para guardar datos que sabemos que siempre van a ser positivos.

4. Variables y tipos de datos primitivos

Tipo con signo	Tipo sin signo
int	uint
long	ulong
short	ushort

sbyte	byte
-------	------

Tipos numéricos de punto flotante

- Los tipos numéricos de punto flotante representan números con decimales, es decir, el conjunto de los números reales.
- Dependiendo del rango que puedan almacenar y la precisión de dígitos decimales que proporcionan tenemos 3 tipos de punto flotante en C#: `float`, `double` y `decimal`.

4. Variables y tipos de datos primitivos

Tipos numéricos de punto flotante

4. Variables y tipos de datos primitivos

Tipo	Tamaño	Rango (aprox)	Precisión	Literal
float	32 bits	1.5×10^{-45} a 3.4×10^{38}	7 dígitos	f, F
double	64 bits	5.0×10^{-324} a 1.7×10^{308}	15-16 dígitos	
decimal	128 bits	-7.9×10^{-28} a 7.9×10^{28}	28 dígitos	m, M

float

- Se utiliza para representar números decimales que no requieren gran precisión. Al ocupar la mitad de espacio que un `double`, ofrece un gran balance entre precisión y eficiencia.
- Para asignar valores a un `float`, **siempre** debemos utilizar el literal `f` o `F` después del valor.

4. Variables y tipos de datos primitivos

```
1 float pi = 3.141f
```

double

- Es el tipo de punto flotante más utilizado en C#.
- Ofrece un rango y una precisión bastante mayores que un float, por lo que suele ser la opción que elegiremos para almacenar números con decimales.

4. Variables y tipos de datos primitivos

```
1 double pi = 3.141592653
```

decimal

- Se utiliza cuando la precisión decimal es crítica, como cálculos físicos u operaciones monetarias. Tiene menor rango que un `double` pero mucha mayor precisión.
- Ocupa 128 bits, lo que supone una carga importante en memoria y procesamiento de cálculo.
- Para asignar valores a un `decimal`, **siempre** debemos utilizar el literal `m` o `M` después del valor.

4. Variables y tipos de datos primitivos

```
1 decimal pi = 2.14159265358979323846
```

Notación científica

- Los 3 tipos de punto flotante en C# admiten notación científica.
- Para expresar un número como potencia de 10, debemos utilizar el carácter E seguido del exponente.
- Por ejemplo, para representar la constante de Plank ($6.62607015 \times 10^{-34}$) utilizaríamos la expresión:

4. Variables y tipos de datos primitivos

```
1 double plankConstant = 6.626070
```

bool

- El tipo `bool` es un tipo primitivo que almacena un valor lógico booleano, es decir, puede representar únicamente los valores verdadero (`true`) o falso (`false`).
- Es un tipo fundamental en la programación ya que nos permite comprobar las condiciones lógicas que controlarán el flujo de un programa.

4. Variables y tipos de datos primitivos

char

- El último de los tipos primitivos de C# es char, que almacena un único **carácter Unicode**.
- Unicode es un estándar de codificación de textos que contiene letras (de la mayoría de los alfabetos del mundo), números (como carácter de su dígito), símbolos y caracteres de control.
- El tipo char ocupa 16 bits e internamente es un número entero, pero se trata de manera diferente debido a los datos que representa.

4. Variables y tipos de datos primitivos

char

- Los valores de tipo char se asignan mediante comillas simples (').
- Los caracteres especiales se representan mediante la barra inclinada (\), como por ejemplo un salto de línea (\n), una tabulación (\t) o la comilla simple (\').
- También podemos asignar caracteres Unicode mediante su código con el carácter especial \u, seguido del código en hexadecimal. Por ejemplo, \u00A9 daría como resultado el carácter '©'.

4. Variables y tipos de datos primitivos

char

- Ejemplos:

- Lista completa de caracteres Unicode:

4. Variables y tipos de datos primitivos

```
1 char letter = 'j';
2 char nubmer = '7';
3 char percent = '%';
4 char percentUnicode = '\u0025';
5 // percent y percentUnicode representan el mi
6 char lineBreak = '\n';
```

<https://www.fileformat.info/info/charset/UTF-16/list.htm>

Tipado fuerte

- Como vimos en el tema anterior, C# es un lenguaje **fuertemente tipado**. Esto significa que cada variable pertenece a un único tipo y este tipo es inmutable en el tiempo.
- Además, a una variable no se le pueden asignar valores de un tipo que no sea compatible con su tipo.
- El **compilador** se encarga de verificar la integridad del tipado (dando error si nos saltamos las reglas).

5. Conversión de tipos

- Ejemplo:
- Estas 2 asignaciones darían como resultado un error y no se permitiría la compilación del programa.
- En el primer caso intentamos asignar un valor decimal a una variable de tipo entero; en el segundo un texto a una variable numérica. Ambas asignaciones son incompatibles.

5. Conversión de tipos

```
1 int integerNumber =  
2  
3 double floatNumber;
```

```
4 floatNumber = "text"
```

Conversión explícita e implícita

- Para poder intercambiar datos entre tipos de variable diferentes debemos convertir el tipo de datos de uno a otro.
- Esta conversión puede realizarse de manera implícita en unos casos concretos y de manera general deberá realizarse de manera explícita.

5. Conversión de tipos

Conversión implícita

- La conversión implícita únicamente se puede llevar a cabo entre tipos primitivos y cuando no implique la posible pérdida de información.
- Esto quiere decir que podremos convertir un tipo de datos en otro siempre y cuando el tipo de destino sea de mayor capacidad que el de origen.
- Por ejemplo, podemos convertir un `int` en un `long` o un `double`, ya que ambos tipos tienen capacidad de representar cualquier valor de `int`, pero no al revés.

5. Conversión de tipos

Conversión implícita

- Para realizar una conversión implícita únicamente debemos asignar el valor de una variable a otra de tipo compatible.
- En este ejemplo, `zipCodeLong` tendría el mismo valor que `zipCode` y `piDouble` tendría el mismo valor que `pi`.

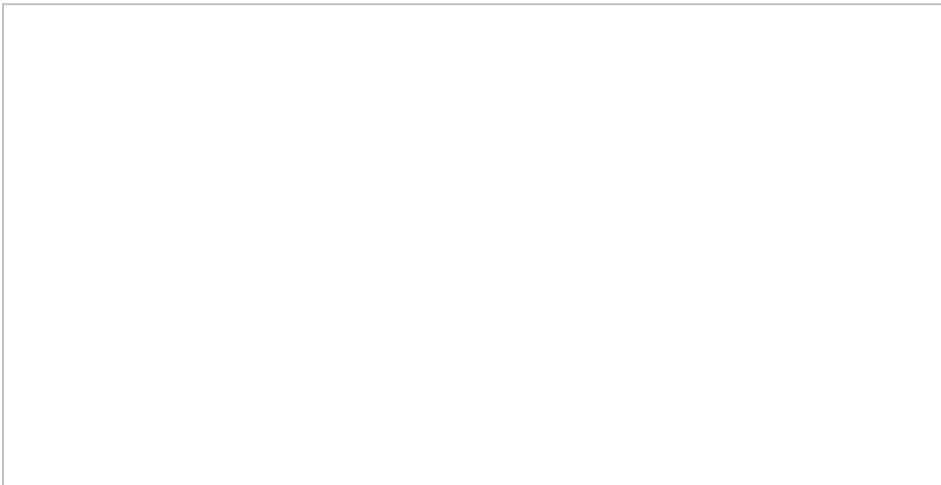
5. Conversión de tipos

```
1 int zipCode = 16002;  
2 long zipCodeLong = zipCode;  
3  
4 float pi = 3.1415927f;
```

Conversión implícita

- Las únicas conversiones implícitas que se pueden realizar pues, son las siguientes (no hace falta que sea conversión al tipo directamente superior, un `short` se puede convertir en un `long` o en un `double` directamente):

5. Conversión de tipos



Conversión explícita

- En C#, siempre que exista **riesgo de pérdida de información**, debemos realizar la conversión de manera explícita.
- Para realizar una conversión explícita realizamos un *casting* que indica que el valor pasa a ser del tipo de destino.
- Este casting se realiza poniendo entre paréntesis el tipo al que se va a convertir un valor o variable.

5. Conversión de tipos

Conversión explícita

- Las conversiones explícitas pueden incurrir en **pérdida de datos o de precisión** debido a que el tipo de destino no tiene la misma capacidad que el de origen.
- Las conversiones entre clases personalizadas siempre deberán de hacerse de manera explícita (entraremos en detalle en el tema 7).

5. Conversión de tipos

Conversión explícita

- Ejemplos:

5. Conversión de tipos

```
1 float pi = (float)3.14159265358
2
3 double debt = -44129.99;
4 short debtShort = (short)debt;
5
6 ulong kilometers = 275806;
7 uint kilometersUInt = (uint)kil
```

Conversión explícita

- En el primer caso realizamos un casting directamente sobre un valor.

Como resultado obtenemos una pérdida de información ya que el tipo `float` no tiene precisión suficiente para almacenar el valor asignado.

Al finalizar la ejecución, la variable `pi` contiene el valor 3.14159274 ya que debido a su precisión no es capaz de ajustar más.

5. Conversión de tipos

Conversión explícita

- En el segundo caso del ejemplo realizamos un casting de tipos muy diferentes. Incurrimos en pérdida de información ya que el tipo `short` por una parte no alcanza al rango del número almacenado en el `double` y por otra no puede representar decimales.
- Debido a las limitaciones intrínsecas del propio tipo de dato, esta conversión nos da como resultado un **valor incoherente**: al finalizar la ejecución `debtShort` tiene asignado el valor 21407.

5. Conversión de tipos

Conversión explícita

- En el tercer caso también realizamos un casting sobre una variable pero esta vez no hay pérdida de información

Aunque el tipo `uint` tenga menor capacidad que el `ulong`, el valor almacenado sí que está dentro del rango de `uint`, por lo que la variable `kilometersUint` almacenará al final de la ejecución el valor 275806.

5. Conversión de tipos

Vídeo práctico

5. Conversión de tipos

5. Conversión de tipos

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int integerNumber = -7518965;
8             long longNumber = 148995119784984;
9             ushort unsignedShortNumber = 11111;
10
11             double decimalNumber = -27.1598498;
12             float floatingNumber = 3.14159625849884;
13
14             bool condition = true;
15
16             integerNumber = (int)longNumber;
17
18             unsignedShortNumber = (ushort)decimalNu
19
20             integerNumber = (int)4865.1684;
21         }
22     }
23 }
```

¿Qué es un operador?

- En C#, los operadores son símbolos que indican una **operación entre operandos** (ya sean estos variables o datos).
- Dependiendo del tipo de operación que realizan, se clasifican en diferentes categorías:
 - Operadores aritméticos
 - Operadores de asignación
 - Operadores de comparación
 - Operadores lógicos
 - Otros tipos de operadores avanzados

6. Operadores

Operadores aritméticos

- Realizan operaciones matemáticas básicas.

6. Operadores

Operador	Símbolo	Descripción	Uso
Suma	+	Devuelve el valor de la suma de 2 operandos	<code>x + 4;</code> <code>3.7 + 4;</code>
Resta	-	Devuelve el valor de la resta de 2 operandos	<code>12 - i;</code> <code>a - b;</code>
Multiplicación	*	Devuelve el valor de la multiplicación de 2 operandos	<code>38.5 * 2;</code> <code>radius * pi;</code>
División	/	Devuelve el valor de la división de 2 operandos	<code>area / 2;</code> <code>27.4 / 3.768;</code>

Módulo	%	Devuelve el resto de la división de 2 operandos	<code>x % y;</code> <code>7.5 % 2;</code> <code>//Devuelve 1,5</code>
Incremento	++	Aumenta en 1 el valor de una variable	<code>i++;</code>
Decremento	--	Decrementa en 1 el valor de una variable	<code>j--;</code>

Operadores de asignación

- Asignan un valor a la variable que los precede (**siempre** tienen que ir precedidos de una).

6. Operadores

Operador	Símbolo	Descripción	Uso	Equivalente
Asignación simple	=	Asigna a la variable el valor del operando de la derecha.	<code>x = 5;</code>	<code>x = 5;</code>
Suma y asignación	<code>+=</code>	Asigna a la variable el valor actual de la variable más el operando.	<code>p += 27;</code> <code>i += j;</code>	<code>p = p + 27;</code> <code>i = i + j;</code>
Resta y asignación	<code>-=</code>	Asigna a la variable el valor actual de la	<code>a -= 9.99;</code>	<code>a = a - 9.99;</code>

		variable menos el operando.		
Multiplicación y asignación	<code>*=</code>	Asigna a la variable el valor actual de la variable multiplicada por el operando.	<code>d *= pi;</code>	<code>d = d * pi;</code>
División y asignación	<code>/=</code>	Asigna a la variable el valor actual de la variable dividida por el operando.	<code>b /= 3;</code>	<code>b = b / 3;</code>
Módulo y asignación	<code>%=</code>	Asigna a la variable el resto de la división entre el valor actual y el operando.	<code>n %= 2;</code>	<code>n = n % 2;</code>

Operadores de comparación

- Comparan 2 operandos y **devuelven un valor bool**.

Para entender mejor el ejemplo, vamos a contar con unas variables inicializadas como 'a = 4' y 'b = 3.5'

6. Operadores

Operador	Símbolo	Descripción	Uso	Valor devuelto
Igual a	==	Devuelve true si ambos operandos tienen el mismo valor.	<pre>a == 4; b == a;</pre>	true false
Distinto de	!=	Devuelve true si los operandos tienen distinto valor.	<pre>a != b; a != 4;</pre>	true false

Mayor que	>	Devuelve true si el operando de la izquierda es mayor que el de la derecha.	a > 10; a > b;	false true
Menor que	<	Devuelve true si el operando de la izquierda es menor que el de la derecha.	b < 4.98; a < 2;	true false
Mayor o igual que	>=	Devuelve true si el operando de la izquierda es mayor o igual que el de la derecha.	b >= a; a >= 2;	false true
Menor o igual que	<=	Devuelve true si el operando de la izquierda es menor o igual que el de la derecha.	b <= 3.5; b <= 3.6;	true false

Operadores lógicos

- Comparan operandos **bool** y devuelven también un **valor bool**.

Para ver mejor el ejemplo, vamos a contar con unas variables inicializadas como '`v = true`' y '`f = false`'

6. Operadores

Operador	Símbolo	Descripción	Uso	Valor devuelto
AND lógico (y)	<code>&&</code>	Devuelve true si ambos operandos son true.	<code>v && f;</code> <code>v && true;</code>	false true
OR lógico (o)	<code> </code>	Devuelve true si uno de los operandos es true.	<code>v f;</code> <code>f false;</code>	true false

NOT lógico (no)	!	Devuelve el valor contrario al operando que va detrás (operando de operador único).	!v; !false ;	false true
--------------------------------	---	--	--------------------	---------------

Otros operadores avanzados

- C# cuenta con más tipos de operadores de uso avanzados, como operadores para trabajar a nivel de bit, operadores para trabajar con tipos, operadores condicionales u operadores de expresión Lambda.
- En este tema no los vamos a ver, pero más adelante conoceremos y trabajaremos con algunos de ellos.

6. Operadores

Asociatividad de operadores

- En C# podemos asociar operadores entre sí, es decir, **concatenar operaciones** compatibles cuyo resultado será la combinación de todas ellas.
- En este sencillo ejemplo, concatenamos un operador de asignación, uno de suma y otro de resta.
- El resultado será la asignación a la variable x del valor de las operación de suma de y con 28.5 menos z.

6. Operadores

`1 x = y + 28.5`

Asociatividad de operadores

- No podremos asociar operaciones no compatibles entre si. Esto significa que no podemos asociar operadores aritméticos con operadores lógicos, por ejemplo.
- La línea anterior daría como resultado un error ya que el operador lógico `||` da como resultado un bool mientras que la suma de un entero necesita otro tipo numérico como segundo operando.

6. Operadores

1 x = y || z + .

Precedencia de operadores

- Al igual que en las matemáticas, en C# también existe la precedencia de operadores.
- Esto quiere decir que cuando asociamos operadores, las operaciones cuyo operador tenga mayor precedencia (o **prioridad**) se resolverán en primer lugar.
- En este ejemplo, la variable a tendrá el valor 9, ya que primero evalúa la multiplicación $2 * 2$ y posteriormente la suma con el resultado de la primera operación.

6. Operadores

1 a = 5 + 2 *

Precedencia de operadores

- El orden jerárquico de los operadores es el siguiente:

6. Operadores

++, --, !
*, /, %
+, -
>, >=, <, <=
==, !=
&&,
=, +=, -=, *=, /=, %=

Precedencia de operadores

- Al igual que en el lenguaje matemático, si necesitamos evaluar una expresión con menor precedencia en primer lugar, utilizaremos paréntesis ().
- En este caso, a tendría el valor 14 ya que se realiza la operación $5 + 2$ en primer lugar y posteriormente la multiplicación.

6. Operadores

1 2 - (5 + 2

Vídeo práctico

6. Operadores

6. Operadores

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             int x = 7, y = -2, z = 10;
8             double a = 78.280, b = 2.0001;
9
10            int addition = x + y;
11            int add2 = x + y + z;
12
13            double result = a - z * b;
14            double res2 = (a - z) * b;
15
16            bool firstCondition = true, secondCondition = x % 2 > 0;
17
18            bool and = firstCondition && secondCondition;
19            bool andFalse = firstCondition && false;
20
21            bool complexBoolOperation = firstCondition && x * z > b / a
22        }
23    }
```


- En C#, un `string` es un objeto cuyo valor es un **texto**. Es por ello que se trata de uno de los tipos de datos más utilizados en la programación.
- Internamente, un `string` almacena una colección de objetos `char` (que ya hemos visto previamente).
- Para asignar valor a un `string`, utilizamos las comillas dobles (") al abrir y cerrar el texto contenido:

7. Cadenas de texto

```
1 string quixote = "En un lugar de La M
```

- Las cadenas en C# también pueden ser nulas o estar vacías.
- Una cadena vacía significa que tiene un valor asignado, pero este es un texto "vacío" (se corresponde al valor ""), mientras que una cadena nula significa que **no tiene valor asignado**. Aprenderemos el significado de `null` en el siguiente tema.
- Por defecto, una cadena será nula hasta que se le asigne un valor.

7. Cadenas de texto

```
1 string texto; // En este punto, la variable 'texto' es nula
2 texto = ""; // Podemos asignar también una cadena vacía
3 texto = "Curso C# 13"; // Aquí ya tendría una cadena de texto
```

- Los strings son objetos **inmutables**. Esto quiere decir que su valor nunca cambia después de su creación.
- Por ello, todos los métodos y operaciones que *aparentemente* modifican un `string`, realmente están devolviendo una instancia nueva con el resultado de la operación. Debemos asignar el resultado de las operaciones si queremos trabajar con él.
- Tenemos a nuestra disposición un amplísimo catálogo de mecanismos para trabajar con cadenas. Vamos a conocer algunos de los más importantes.

7. Cadenas de texto

Concatenación de cadenas

- Son operaciones que concatenan las diferentes cadenas involucradas, de izquierda a derecha. Se puede realizar mediante varios métodos:
- **+**: Devuelve la concatenación las cadenas que aparecen a la izquierda y a la derecha del operador.
- **+=**: **Asigna** a la cadena de la izquierda del operador la concatenación de esta con la cadena de la derecha del operador.
- **string.Concat(s1, s2, s3...)**: Devuelve la concatenación de **todas las cadenas** que se encuentran dentro de los paréntesis.

7. Cadenas de texto

Concatenación de cadenas

7. Cadenas de texto

```
1  string start = "Cadena inicial";
2  string end = "Cadena final";
3
4  string result1 = start + " con añadido";
5  // result1 almacena "Cadena inicial con añadido"
6
7  string result2 = start + end;
8  // result2 almacena "Cadena inicialCadena final"
9
10 string result3 = string.Concat(start, ", ", end, ".");
11 // result3 almacena "Cadena inicial, Cadena final."
12
13 start += " más larga";
14 // start ahora almacena "Cadena inicial más larga"
15
16 start + end + ".";
17 // NO SE HA ASIGNADO LA OPERACIÓN por lo que ninguna variable almacena
```

Concatenación de cadenas

- También podemos concatenar cadenas mediante el método `string.Format`.
- Este nos proporciona un potente pero sencillo mecanismo para incorporar variables a una cadena mediante un sistema de índices.
- Su sintaxis es:

7. Cadenas de texto

```
1 string.Format("Intercalo el objeto {0} y después el {1}")
```

Concatenación de cadenas

- Ejemplo:

- Este método sustituye cada {x} por el argumento en la posición x detrás de la cadena.

7. Cadenas de texto

```
1 int birthYear = 1881;  
2 string name = "Pablo Picasso";  
3  
4 string.Format("El pintor {0} nació en {1} ", name, birthYear);
```



Comparación de cadenas

- Operaciones que comparan cadenas y devuelven un valor booleano:
- **`string.Equals(s1, s2)`**: Devuelve `true` si el texto contenido en `s1` es **igual** al de `s2`. Tiene en cuenta mayúsculas, minúsculas, espacios, signos de puntuación, etc. Deben ser exactamente iguales para que sea `true`.
- **`string.IsNullOrEmpty(s1)`**: Devuelve `true` si la cadena `s1` **no está asignada** o tiene asignada una **cadena vacía** (`""`).

7. Cadenas de texto

Comparación de cadenas

7. Cadenas de texto

```
1  string s1 = "Texto";
2  string s2 = "";
3
4  bool areEquals1 = string.Equals(s1, "Texto");
5  // areEquals1 es true
6
7  bool areEquals2 = string.Equals(s1, "texto");
8  // areEquals2 es false porque tiene en cuenta las
9
10 bool isEmpty = string.IsNullOrEmpty(s2);
11 // isEmpty es true
```

Búsqueda en cadenas

- Operaciones que buscan información en el texto de una cadena.
- `s1.Contains(s2)`: Se invoca sobre una variable (`s1`) y devuelve `true` si esta **contiene** el texto de la variable entre paréntesis (`s2`).
- `s1.StartsWith(s2)`: Devuelve `true` si la variable `s1` **empieza** exactamente con el contenido de la variable `s2`.
- `s1.EndsWith(s2)`: Devuelve `true` si la variable `s1` **acaba** exactamente con el contenido de la variable `s2`.

7. Cadenas de texto

Búsqueda en cadenas

7. Cadenas de texto

```
1  string title = "Curso C# 13 - Tema 2";
2  string course = "Curso";
3  string topic = "tema 2";
4
5  bool containsLanguage = title.Contains("C# 13");
6  // containsLanguage es true
7
8  bool startWithCourse = title.StartsWith(course);
9  // startWithCourse es true
10
11 bool endsWithTopic = title.EndsWith(topic);
12 // endsWithTopic es false porque distingue mayúsculas y
```

Transformaciones de cadenas

- Operaciones para modificar el valor que contiene una cadena.
- `s1.Replace(sub1, sub2)`: Devuelve la cadena resultante de **reemplazar** todas las repeticiones que aparecen en `s1` del texto contenido en `sub1` por el texto de `sub2`.
- `s1.ToUpper()`: Devuelve una cadena con el contenido de `s1` convertido a **mayúsculas**.
- `s1.ToLower()`: Devuelve una cadena con el contenido de `s1` convertido a **minúsculas**.

7. Cadenas de texto

Transformaciones de cadenas

7. Cadenas de texto

```
1  string text = "Curso C# 13 - Cadenas en C#  
2  
3  string java = text.Replace("C#", "Java");  
4  // java almacena "Curso Java 13 - Cadenas  
5  
6  string upper = text.ToUpper();  
7  // upper almacena "CURSO C# 13 - CADENAS E  
8  
9  string lower = text.ToLower();  
10 // lower almacena "curso c# 13 - cadenas e
```

Subcadenas

- Operadores que devuelven partes contenidas en cadenas de diferentes formas.
 - `s1.Substring(start, length)`: Devuelve una nueva cadena con los '*length*' siguientes caracteres a partir de la posición '*start*' de `s1`. Se puede no especificar `length` y cogerá todos los caracteres hasta el final.
- *¡IMPORTANTE! En C# siempre se empieza a contar por 0. Lo veremos más adelante.*
- `s1.Trim()`: Devuelve una cadena que elimina los espacios en blanco al inicio o final de `s1`.

7. Cadenas de texto

Subcadenas

7. Cadenas de texto

```
1  string fullName = "  Manuel Ruiz  ";
2
3  string name = name.Substring(2, 6);
4  // name almacena "Manuel"
5
6  string surname = name.Substring(8);
7  // surname almacena "Ruiz  "
8
9  string cleanedName = name.Trim();
10 // cleanName almacena "Manuel Ruiz"
```

Consultas a cadenas

- Operaciones que sirven para consultar diversa información sobre una cadena:
- `s1.Length`: Devuelve un `int` con la longitud de la cadena.
- `s1.IndexOf(s2)`: Devuelve un `int` con la primera aparición de la cadena `s2` dentro de `s1`. Si no está contenida, devuelve `-1`. También podemos utilizar `LastIndexOf` para encontrar la última aparición.
- `s1[pos]`: Devuelve un `char` con el caracter que se encuentra en la posición '*pos*' de `s1`.

**(Recordemos que el primer caracter es 0, no 1).*

7. Cadenas de texto

Consultas a cadenas

7. Cadenas de texto

```
1  string salutation = "¡Hola, Don Pepito!";
2
3  int salutationLenght = salutation.Length;
4  // saludoLenght tiene el valor 18
5
6  int oPosition = salutation.IndexOf("o");
7  // oPosition tiene el valor 2
8
9  int lastOPosition = salutation.LastIndexOf
10 // oPosition tiene el valor 16
11
12 char sixthChar = salutation[5];
13 // sixthChar almacena el caracter ','
```

Vídeo práctico

7. Cadenas de texto

7. Cadenas de texto

```

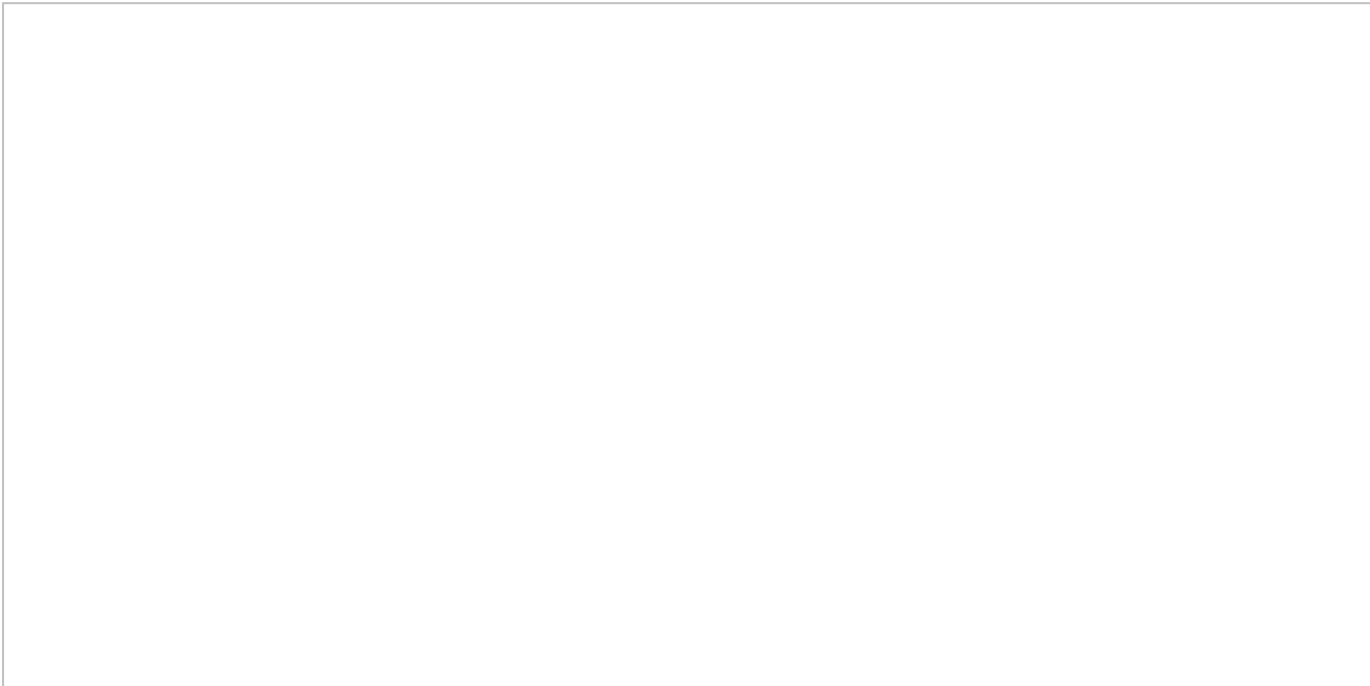
1  namespace ConsoleApp1
2  {
3      internal class Program
4      {
5          static void Main(string[] args)
6          {
7              string title = "El ingenioso hidalgo Don Quijote
8              string character = "Don Quijote";
9              string location = "    la mancha    ";
10             string emptyString = "";
11
12             string fullCharacter = character + " de La Mancha
13             string.Concat("Mi personaje es ", fullCharacter);
14
15             bool isEmpty = string.IsNullOrEmpty(emptyString);
16             bool isEmpty2 = string.IsNullOrEmpty(fullCharacte
17
18             bool containsCharacter = title.Contains(character
19             bool containsHidalgo = title.Contains("Hidalgo");
20             bool containsHidalgo2 = title.Contains("Hidalgo".
21
22             bool startsWith = location.StartsWith(" ");
23             bool endsWith = location.EndsWith("mancha");
24
25             Console.WriteLine("Title: {0}", title);

```

¿Qué es la consola?

- Llamamos consola a la ventana de línea de comandos del sistema operativo.

8. Trabajando con la Consola



¿Qué es la consola?

- Se trata de un mecanismo sencillo pero fundamental para interactuar con el usuario ya que nos permite tanto mostrar información como recibirla a través de líneas de texto.
- Para trabajar con la consola en C# usaremos la clase `Console`.

8. Trabajando con la Consola

Salida de datos

- Denominamos salida de datos al hecho de escribir cualquier texto por pantalla.
- Para escribir texto en consola utilizaremos los métodos `Write` y `WriteLine`.
- `Console.Write(s1)`: Escribe el texto contenido en `s1` en la posición actual del cursor.
- `Console.WriteLine(s1)`: Escribe el texto y además cambia el cursor a la siguiente línea.

8. Trabajando con la Consola

Salida de datos

8. Trabajando con la Consola

```
1 string salutation = "Hola c  
2  
3 Console.Write(salutation);  
4 Console.Write("123");  
5  
6 Console.WriteLine(salutation  
7 Console.WriteLine(salutation  
8 Console.WriteLine(salutation
```

Entrada de datos

- Denominamos entrada de datos al hecho de introducir datos en la consola mediante nuestro teclado.
- Para leer entradas de texto en nuestro programa utilizaremos `ReadLine`:
- `Console.ReadLine()`: El programa queda a la espera de que el usuario introduzca cualquier texto. Este texto durará hasta que pulse la tecla '*Enter*'.

8. Trabajando con la Consola

Entrada de datos

- En este ejemplo escribimos por pantalla en primer lugar.
- Después leemos una entrada que asignamos a la variable `input`.
- Por último, escribimos una nueva salida en la que concatenamos texto con el contenido de la variable que acabamos de leer.

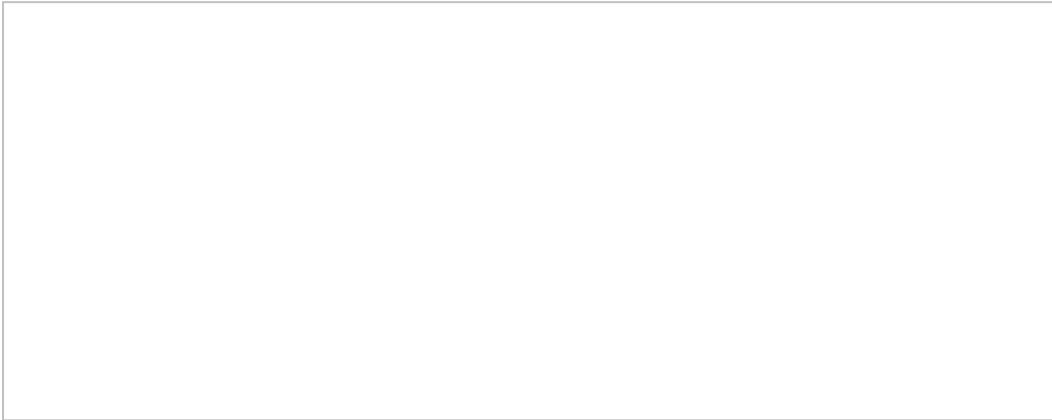
8. Trabajando con la Consola

```
1 Console.Write("Introduzca texto: ");  
2 string input = Console.ReadLine();  
3 Console.WriteLine(input + " ");
```

Entrada de datos

- El resultado del ejemplo anterior es el siguiente:

8. Trabajando con la Consola



Entrada de datos

- La entrada de datos por consola siempre devuelve un valor de tipo `string`.
- Si queremos trabajar con valores numéricos, debemos utilizar las funciones `Parse` de los tipos numéricos que hemos visto:
- `int.Parse(s1)`: Devuelve el valor numérico del texto escrito en `s1` si este es un número. Si no introducimos como texto un número válido para el tipo, obtendremos un error de ejecución.

Aprenderemos más sobre errores y excepciones más adelante.

8. Trabajando con la Consola

Entrada de datos

8. Trabajando con la Consola

```
1  int inputInteger = int.Parse("53");
2  // inputInteger vale 53
3
4  double inputDouble = double.Parse("4710.820");
5  // inputDouble vale 4710.82
6
7  float inputText = float.Parse("53.texto");
8  // ERROR, el texto no es un número válido
9
10 short inputShort = short.Parse("480201");
11 // ERROR, el número introducido es más grande que el rar
12
13 int inputDecimals = int.Parse("53.75");
14 // ERROR int no soporta decimales
```

Personalización de la consola

- `Console.Clear()`: Limpia todo el texto que hay en ese momento.
- `Console.Title = s1`: Cambia el título de la ventana al texto contenido en `s1`.
- `Console.BackgroundColor=ConsoleColor.XXX`: Cambia el color del fondo al especificado `XXX`.
- `Console.ForegroundColor=ConsoleColor.XXX`: Cambia el color del texto al especificado `XXX`.

(ConsoleColor contiene una lista de colores, os aparecerá en Visual Studio)

8. Trabajando con la Consola

Personalización de la consola

- `Console.ResetColor()`: Vuelve a los colores por defecto.
- `Console.WindowWidth = width`: Cambia el ancho de la consola a *width* caracteres. Debe ser un entero.
- `Console.WindowHeight = height`: Cambia el alto de la consola a *height* líneas. Debe ser un entero.

8. Trabajando con la Consola

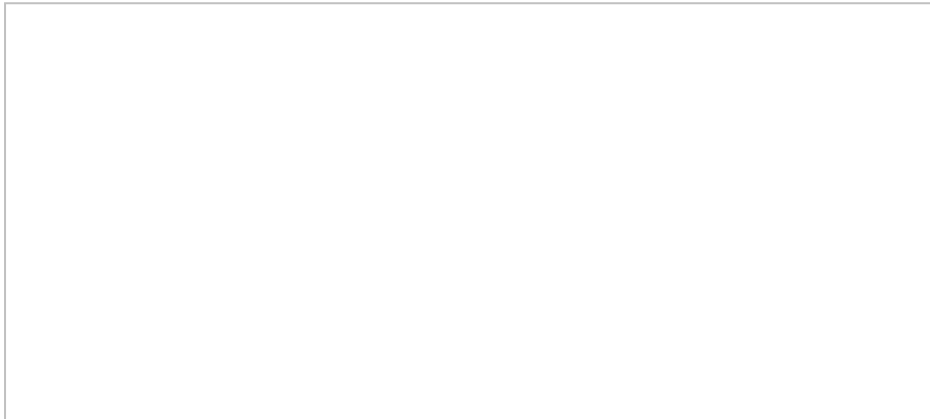
**En las versiones nuevas de Windows, el tamaño de la ventana no cambia,*

pero sí el tamaño que ocupa la consola dentro de esa ventana.

Personalización de la consola

8. Trabajando con la Consola

```
1 Console.Title = "Mi consola personalizada";  
2 Console.Width = 48;  
3 Console.Height = 8;  
4 Console.BackgroundColor = ConsoleColor.Yellow;  
5 Console.ForegroundColor = ConsoleColor.DarkBlue;  
6 Console.Clear();  
7  
8 Console.WriteLine("Curso de C# 13 - Consola person
```



Vídeo práctico

8. Trabajando con la Consola

8. Trabajando con la Consola

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.ForegroundColor = ConsoleColor.Green;
8             Console.Write("*****");
9             Console.Write(" CURSO C# 13 ");
10            Console.WriteLine("*****");
11
12            Console.Write("Introduce tu nombre: ");
13            string name = Console.ReadLine();
14            Console.WriteLine("!Hola " + name + "!");
15
16            Console.Write("Introduce un número: ");
17            string numberString = Console.ReadLine();
18            double numberDouble = double.Parse(numberString);
19            Console.WriteLine("El doble de " + numberString + " es " +
20                2 * numberDouble);
21        }
22    }
```

¿Qué es el código de estilo?

- En programación, el código de estilo se refiere a un **conjunto de reglas** y convenciones que se recomienda seguir al escribir código.
- Este código de estilo no es único (cada lenguaje tiene el suyo recomendado e incluso dentro del mismo lenguaje cada organización puede tener su propia variación) y **no afecta al funcionamiento** del programa. Sin embargo, es muy importante seguirlo para obtener código de calidad y mantenible.

9. Código de estilo

Importancia del código de estilo

- **Mejora la legibilidad:** El código de calidad es más fácil de entender.
- **Facilita el mantenimiento:** Código bien organizado y claro es más fácil de modificar y depurar.
- **Favorece la consistencia:** Siguiendo las mismas reglas siempre, el código es homogéneo a lo largo del tiempo.
- **Mejora la colaboración:** Los equipos pueden trabajar mejor cuando el estilo es el mismo para todos los miembros.
- **Minimiza errores:** Al seguir prácticas estándar, es más difícil introducir errores comunes.

9. Código de estilo

Nomenclatura

- Se utiliza **PascalCase** en clases, métodos y campos públicos. Comienza con mayúscula y las palabras consecuentes también inician con mayúscula.
 - Ej: `class CarEngine, void FindNextStep()`.
- Se utiliza **camelCase** en variables locales. La primera palabra comienza en minúscula y las siguientes con mayúscula. Si la variable es privada, además comenzará con guión bajo.
 - Ej: `string fullName;`
`private double _cornerRadius;`

9. Código de estilo

Estructura

- Todos los bloques de código se abren con una llave ({), que va en una línea ella sola. Se cierran con una llave (}) en una línea ella sola al mismo nivel al que se abrió.
- El código que va dentro de un bloque se indenta un nivel hacia adentro. El indentado son 4 espacios.
- No hay espacio al abrir o cerrar paréntesis, pero sí hay entre un operando y sus operandos.

9. Código de estilo

Estructura

9. Código de estilo

```
1  public class NumberCalculations
2  {
3      private int _counter;
4
5      public bool IsOddNumber(int
6      {
7          if(input % 2 == 0)
8          {
9              return true;
10         }
11         else
12         {
13             return false;
14         }
15     }
```

Longitud de línea

- Las líneas de código no deberían tener más de 80-100 caracteres. Si una línea es muy larga y se puede dividir, es mejor dividirla (se pueden introducir saltos de línea entre operadores, por ejemplo, y hasta que no escribamos el ; seguirá siendo la misma sentencia).

9. Código de estilo

```
1 double veryComplexCalculus = 2 * radius  
2    - (initialMomentum / earthOrbit) * 0
```

Otras consideraciones

- Los nombres de clases, variables y métodos deben ser **descriptivos** e informarnos de su propósito. Un buen nombre para una variable sería `double price`, mientras que `int number` sería un mal nombre.
- Es muy recomendable utilizar el **idioma inglés** para poner nombres en nuestro código. Podemos utilizar comentarios en castellano si queremos, pero para el código deberíamos utilizar el inglés.

9. Código de estilo

- En este último apartado hemos visto puntos que están fuera del alcance de lo explicado hasta el momento.
NO TE PREOCUPES, es normal que no los entiendas al ver el código, los iremos viendo en temas posteriores.
- Sin embargo, **sí es importante** que te familiarices con el código de estilo desde el principio y que cuando hagas los ejercicios escribas tu código cumpliendo estas reglas.
- Puedes ver las recomendaciones básicas de Microsoft para escribir código C# en el siguiente enlace:

9. Código de estilo

<https://github.com/dotnet/runtime/blob/main/docs/coding-guidelines/coding-style.md>

En este tema hemos aprendido la estructura de un programa en C#, algunos de los tipos de datos más básicos que forman parte del lenguaje y la conversión entre ellos. Por último, hemos aprendido a trabajar con la consola para crear aplicaciones interactivas.

2. Sintaxis básica de C#

Speaker notes